

Higher-Order Metaphysics

Week One: the syntax of higher-order languages

CIAN DORR

(New York University: cian.dorr@nyu.edu)

13th September 2023

Minimal goal for today: knowing the syntax of a higher-order languages—
i.e. what their well-formed expressions are.

More ambitious goal for today: an *understanding* of such a language.

I'll assume a prior familiarity with, and understanding of, first-order languages, and go through the extra elements in several steps.

1 Types

When we are studying formal languages that extend first-order logic, it becomes useful to have a general system of nomenclature for the syntactic categories of the expressions we may find ourselves introducing. The standard approach uses special strings called *types* to label the syntactic categories. The most familiar such set is the set of so-called “simple types”, defined as follows:

Definition (Simple Types). The set of *simple types* is the smallest set of strings that contains the letter e and the letter t , and is such that whenever it contains two strings σ and τ , it also contains the string $(\sigma \rightarrow \tau)$ —the result of writing an arrow between σ and τ and surrounding the resulting string with parentheses.

In referring to simple types, we will allow ourselves to omit parentheses, restoring them from right to left: for example we will write ' $t \rightarrow e \rightarrow t$ ' instead of ' $(t \rightarrow (e \rightarrow t))$ '.

Simple types are useful for labelling syntactic categories, including some syntactic categories already encountered in first-order languages.

- An expression of type t is a *formula* (a *sentence* if it lacks free variables)
- An expression of type e is a “singular term”.¹
- An expression of type $\sigma \rightarrow \tau$ is something that combines with one of type σ to make one of type τ . So, for example, in first order languages, the negation symbol $\neg$ is of type $t \rightarrow t$: it combines with a sentence P to make a new sentence $\neg P$. A monadic predicate F is of type $e \rightarrow t$: it combines with a singular term a to make a sentence Fa .

¹ I mean the things normally just called ‘terms’ in expositions of first-order logic. We’ll be using ‘term’ more widely as a synonym of ‘expression’ (of a given formal language).

But what about the conjunction symbol $\wedge$, or a dyadic predicate R ?

Using a convenient trick (pioneered by Frege (1893) and further developed by Schönfinkel and Curry and Feys (1958)), we can assign simple types to these by thinking of them as taking their arguments one at a time rather than all together. For example, in a conjunctive sentence $P \wedge Q$, we can think of the combination of the first sentence P with the conjunction symbol $\wedge$ not as a meaningless string of symbols, but rather as a well-formed expression of type $t \rightarrow t$ (just like $\neg$), which is combined with Q to make the sentence. $\wedge$ itself will thus be of type $t \rightarrow t \rightarrow t$. (To keep the official syntax simple, we can treat the infix notation $P \wedge Q$ as an unofficial way of writing the string $((\wedge P)Q)$, which explicitly displays the order of pairwise syntactic combination.)

- Similarly, one can think of a binary (first-order) predicate R as having type $e \rightarrow e \rightarrow t$, with Rab short for $((Ra)b)$. More generally, an n -ary predicate will have type $e \rightarrow \dots \rightarrow e \rightarrow t$, with n e 's.

This is reminiscent of what we see in natural languages. In the English sentence ‘Phobos orbits Mars’, ‘orbits Mars’ is a meaningful unit in its own right—a verb phrase (VP), just like ‘rotates’ in ‘Phobos rotates’.

Nevertheless, some might find it suspicious, and want to make room for “primitively polyadic” expressions. They can instead use a different type-system, e.g.:

Definition (Polyadic simple Types). The set of *polyadic simple types* is the smallest set of strings that contains the letter e and the letter t , and is such that whenever it contains $\sigma_1, \dots, \sigma_n$ and τ , it also contains the string $(\sigma_1, \dots, \sigma_n \rightarrow \tau)$.²

I’ll mostly be following Bacon in sticking to simple types, which give a less cumbersome syntax.

² Note that if we allow the case where $n = 0$, this will give us interesting but weird types like $(\rightarrow t)$.

2 Syntax of applicative languages

The following makes sense for any notion of “type”:

Definition. A *T-typed collection* C is a set together with a function assigning each element of that set to an element of T .

We write $x :_C \sigma$ to mean that x is an element of the underlying set of C that is mapped by the function to $\sigma \in T$. C^σ is $\{x \mid x :_C \sigma\}$. We can talk and think about typed collections in the way we are used to thinking about sets—e.g. $C \subseteq D$ iff $x :_D \sigma$ whenever $x :_C \sigma$.

Definition. A *T-typed signature* Σ is a *T-typed collection* of strings (which we call the ‘constants’ of Σ).³

Definition. When Σ is a *T-typed collection*, $\mathcal{A}(\Sigma)$ —the *applicative lan-*

³ To be rigorous, we should require that the constants not contain certain special characters like the parentheses, which would disrupt the unique readability for terms in the corresponding applicative language.

guage over Σ —is the smallest typed collection $*$ such that:

- $c :_* \sigma$ whenever $c :_{\Sigma} \sigma$.
- $(AB) :_* \tau$ whenever $A :_* \sigma \rightarrow \tau$ and $B :_{\varphi} * \sigma$.

Elements of $\mathcal{A}(\Sigma)$ are called (applicative) *terms*. When writing them out we omit parentheses *starting from the left*: ABC is short for $((AB)C)$.⁴

This particular convention for writing out terms means we may have to stare pretty hard at a term to find out its type. If we don't mind taking up a lot of space on the page, we could adopt an alternative 'tree' type notation for terms on which their type is written right into them. For example, if $A :_{\Sigma} \rho \rightarrow \sigma$, $B :_{\Sigma} \sigma \rightarrow \tau$, and $C :_{\Sigma} \rho$, then insted of $B(AC)$ we could write the following:

$$\frac{\frac{\frac{}{\sigma \rightarrow \tau} B \quad \frac{\frac{}{\rho \rightarrow \sigma} A \quad \frac{}{\rho} C}{\sigma}}{\tau}}$$

For this alternative notation, we would define $\mathcal{A}(\Sigma)$ to be the smallest typed collection $*$ (of diagrams) such that

- $\frac{}{\sigma} c :_* \sigma$ whenever $c :_{\Sigma} \sigma$
- $\frac{A \quad B}{\tau} :_* \tau$ whenever $A :_* \sigma \rightarrow \tau$ and $B :_* \sigma$.

We can go back and forth freely between the two ways of notating terms.⁵

Expressions of different types can generally not be substituted for one another grammatically. It is worth emphasizing that, contrary to the impression given by many authors (and to some extent by Russell (1908)), these grammatical restrictions are not something invented to block some paradox that would arise in a system without them. They are motivated in exactly the same way as the grammatical restrictions we are already familiar with in first-order logic, such as the fact that the negation symbol needs to combine with a formula, whereas an ordinary one-place predicate needs to combine with a singular term.

3 The intelligibility of higher-order predicates

- A *predicate type* is any simple type of the form $\sigma_1 \rightarrow \dots \rightarrow \sigma_n \rightarrow t$, with $n \geq 0$
- A *higher-order predicate type* is any type of this form where at least one σ_i is not e .

⁴ There's an obvious notion of one term being a *subterm* of or *contained in* another. Also, every applicative term has a unique *main constant*: a constant is its own main constant, and the main constant of (AB) is that of A . It is easy to see that the type of every subterm of an applicative term is contained in the type of its main constant.

⁵ That's because we can prove *unique readability* for terms in the compact notation: if a string (AB) is a term, then for any terms C and D , if $(AB) = (CD)$, then $C = A$ and $D = B$. This depends on not allowing parentheses to occur as part of the constants.

- A *strictly higher-order predicate type* is any type of this form where at least one σ_i is neither e nor t .
- A *relational type* is any simple type that is either e or a predicate type.

If one wants to introduce a signature of constants with a view to using its formal applicative language to ask questions about the world, obviously one has to do some work to avoid dangers like meaninglessness and excessive vagueness. But is there some special reason to regard *higher-order* predicates as worrisome in these respects in a way that first-order predicates aren't?

Well, note that the logical constants $\neg, \wedge, \vee, \rightarrow$ are higher-order predicates by the above definition. And no-one seems to think that, e.g., $\neg P$ has to be understood as the result of first, somehow, converting the sentence P into a singular term and then making that the argument of a first-order predicate. There is an established enterprise of studying languages that add other constants of the same types as these, in which it is hard to see why one would think that such a “conversion” to first-order logic was needed. *Modal* logic considers languages that add one or more new constants of type $t \rightarrow t$; generalizations like conditional logic add constants taking further type- t arguments (e.g. of type $t \rightarrow t \rightarrow t$). Of course, they need to be explained; if we do so by leveraging our understanding of English words like ‘necessarily’, they will inherit any ambiguity or context-sensitivity that may be present in those words. But this is just the usual situation with any kind of formalization.

What about strictly higher-typed predicates? Well, Frege famously had the idea that the first-order quantifiers should be understood as higher-order predicates—in our system, predicates of type $(e \rightarrow t) \rightarrow t$. In the English ‘everything sparkles’, ‘sparkles’ functions as the argument of ‘everything’. I think this is a precious insight!

Given this basis it is hard to see any principled a priori reason for scepticism about attempts to introduce other strictly higher-order predicates. One good example of a case where this might be fruitful is talk of *meaning* or *reference*—philosophically interesting topics where the kind of rigor made possible by formalization will be particularly welcome. A natural formalization will provide, for each type σ , an “expressing” predicate E_σ of type $e \rightarrow \sigma \rightarrow t$, intended to be the analogue for type σ of the first-order reference predicate in “Mars’ refers to Mars’. With the aid of these predicates (along with a quotational device for talking about linguistic expressions), we will be able to formalize not only claims about meaning in formal languages, but also claims about meaning in natural languages like ‘is self-identical’ $E_{e \rightarrow t} \lambda x^e . x = x$, ‘everything’ $E_{(e \rightarrow t) \rightarrow t} \lambda X^{e \rightarrow t} . \forall y^e . Xy$, and ‘everything is self-identical’ $E_t \forall x^e . x = x$.

4 Forming monadic first-order predicates by abstraction

Natural languages provide a rich array of generating complex predicates. Among verb phrases (VPs), we not only have ‘orbits Mars’, but also ‘doesn’t orbit Mars’, ‘orbits some planet’, ‘orbits itself’, ‘is orbited by Phobos’, and ‘is orbited by something’. While the applicative languages based on a signature of first-order predicates will contain some complex predicates—namely, those formed by combining an n -ary predicate with fewer than n arguments—it does not yet provide analogues of the other complex VPs above. It is natural to try to remedy that by adding a more flexible device for forming complex predicates. The standard such approach is to add *lambda predicates*, constructed according to the following rule:

If P is a formula (expression of type t) and x is an individual variable, $(\lambda x.P)$ is a monadic predicate (expression of type $e \rightarrow t$).

The following glosses will help illuminate the intended interpretations of these predicates (though of course in the light of the remarks in the Introduction you shouldn’t take these as stipulative perfect equivalences):

$(\lambda x. \text{orbits } x \text{ mars})$	orbits Mars
$(\lambda x. \neg(\text{orbits } x \text{ mars}))$	doesn’t orbit Mars
$(\lambda x. \exists y. \text{planet } y \wedge \text{orbits } x y)$	orbits some planet
$(\lambda x. \text{orbits } x x)$	orbits itself
$(\lambda x. \text{orbits phobos } x)$	is orbited by Phobos
$(\lambda x. \exists y. \text{orbits } y x)$	is orbited by something
$(\lambda x. \text{orbits phobos mars})$	is such that Phobos orbits Mars

Having introduced these new predicates, we will need some way of reasoning with them. For example, we want to be able to derive $\text{orbits mars phobos}$ from $(\lambda x. \text{orbits } x \text{ mars}) \text{ phobos}$ and vice versa. The usual such approach is to lay down a rule or axiom of *beta-conversion* that lets us freely interchange any expression of the form $(\lambda v.A)b$ (“beta-redex”) with its “reduct” $A[b/v]$ —the result of substituting b for every free occurrence of the variable v in the expression A —provided that this will not result in any of b ’s free variables becoming bound. (Occurrences bound by lambda are not free.) It is worth noting that with this rule, every sentence involving predicates formed by lambda-abstraction can be proved to be equivalent to some sentence involving no such predicates, suggesting that this does not look like a promising place for someone to dig in their heels and plead unintelligibility.

In the setting where we have added nothing to first-order logic besides complex predicates, the effect of the β rule can also be got from an axiom-

scheme whose instances are biconditionals:

$$\text{Extensional } \beta \quad (\lambda v_1 \dots v_n. P) a_1 \dots a_n \leftrightarrow P[a_1/v_1, \dots, a_n/v_n]$$

where $v_1 \dots v_n$ are all distinct, and any free variables in a_i are safe for v_i in P . However, once we add further resources to the languages, such as modal operators, β becomes strictly stronger.

Once we have introduced this way of forming complex predicates, we can properly implement Frege's idea that quantifiers are higher-order predciaters, by redoing the syntax of first-order logic in such a way that λ is the *only* variable-binder and $\forall$ and $\exists$ are logical constants of type $(e \rightarrow t) \rightarrow t$ (see Stalnaker 1977). We can treat $\forall v.P$ and $\exists v.P$ as shorthand for $\forall(\lambda v.P)$ and $\exists(\lambda v.P)$.

5 Abstraction in general

Our applicative languages already contain some *complex* higher-order predicates—for example, for any formula P , $\wedge P$ is a complex higher-order predicate of type $t \rightarrow t$. But our ability to generate such predicates is quite limited in a way that doesn't seem to correspond to any limit on intelligibility. So, it's natural to generalize our new λ device in a way that lets us generate complex terms of types other than $e \rightarrow t$.

The way Bacon does it is to introduce a stock of variables where each variable has a built in type (Var is a typed collection), and there are infinitely many variables with each type.⁶

Definition. When Σ is a T -typed signature, $\mathcal{L}(\Sigma)$ —the *typed lambda language* over Σ —is the smallest typed collection $*$ such that:

- $c :_* \sigma$ whenever $c :_\Sigma \sigma$.
- $v :_* \sigma$ whenever $v :_{Var} \sigma$.
- $(AB) :_* \tau$ whenever $A :_* \sigma \rightarrow \tau$ and $B :_* \sigma$.
- $(\lambda v.A) :_* \sigma \rightarrow \tau$ whenever $A :_* \tau$ and $v :_{Var} \sigma$.

When writing out terms, we collapse multiple lambdas into one: $\lambda v_1 \dots v_n. A$ is short for $(\lambda v_1. (\lambda v_2. (\dots (\lambda v_n. A) \dots)))$.⁷

Allowing complex higher-order predicates built up by lambda-abstraction lets us introduce new binary connectives like the Scheffer stroke ('neither/nor')

$$\lambda p^t q^t. \neg p \wedge \neg q.$$

new restricted quantifiers

$$\lambda X^{e \rightarrow t}. \forall z^e. \text{red } z \rightarrow Xz$$

⁶ In this sort of formalism it is sometimes natural to indicate the type of a variable with a superscript on some of its occurrences, though figuring it out may often be left as an exercise for the reader

⁷ The set of *free variables* in a term, $FV(A)$, is defined by $FV(c) = \emptyset$, $FV(v) = \{v\}$, $FV(AB) = FV(A) \cup FV(B)$, $FV(\lambda v.A) = FV(A) \setminus \{v\}$.

and new higher-order predicates of the same type as the quantifiers

$$\lambda X^{e \rightarrow t}. X \text{ mars}$$

In fact, by successive application of λ , we can form closed terms in every predicate type.

A variant of this approach—more common in the non-philosophical literature—takes variables to just be strings (without built-in types), that are assigned types as needed. On this approach, there isn't a once-and-for all answer to the question 'is this string a term, and if so what is its type?'. Rather, a string A is only a term of a certain type *relative to an environment* Γ , where Γ is a typed collection of variables (one that assigns a type to each variable free in A). Most often in this approach, lambda abstractions are required to declare a type for the abstracted variable: one writes $(\lambda v : \sigma. A)$ or $(\lambda v^\sigma. A)$.

If we aren't trying to conserve space, a 'tree' notation that displays the type of the term and all its subterms and variables may be helpful. For example, if $x :_{Var} \sigma$, $y :_{Var} \sigma$, $R :_\sigma \sigma \rightarrow \sigma \rightarrow \tau$ and $A :_\Sigma \sigma$, then instead of $(\lambda x. Rx A)y$, one would write

$$\frac{\frac{\frac{\frac{}{\sigma \rightarrow \sigma \rightarrow \tau} R}{\sigma \rightarrow \tau} [\sigma]^x}{\sigma \rightarrow \tau} \text{App} \quad \frac{}{\sigma} A}{\sigma \rightarrow \tau} \text{App} \quad \frac{\frac{\frac{}{\sigma \rightarrow \tau} \lambda x}{\sigma \rightarrow \tau} (\sigma)^y}{\tau} \text{App}$$

6 Higher-order quantifiers and identity.

The last (and probably most controversial) step is to add new logical constants, intended to stand to higher types just as the usual identity symbol and quantifiers of first-order logic stand to type e . In particular: for each type σ , we have new logical constants $=_\sigma$ of type $\sigma \rightarrow \sigma \rightarrow t$, and $\forall_\sigma$ and $\exists_\sigma$ of type $(\sigma \rightarrow t) \rightarrow t$.⁸

For certain types $\sigma \neq e$, we can find expressions in natural languages which look *prima facie* like close analogues to the quantifiers and identity of type σ . For higher-order quantification, Prior (1971) considers sentences like 'He is something I am not—kind' and 'However he says things are, thus they are'.⁹ For higher-order identity, Dorr (2016) considers sentences like 'To be a water molecule is to a molecule consisting of two hydrogen atoms and one oxygen atoms' and 'For there to be vixens is for there to be female foxes'. Such natural-language counterparts provide one potentially helpful entry-point for a more general understanding of higher-order quantification and identity.

⁸ As in the first-order case, $\forall v^\sigma. P$ abbreviates $\forall_\sigma(\lambda v^\sigma. P)$; $\forall u^\sigma v^\tau. P$ abbreviates $\forall_\sigma(\lambda u^\sigma. \forall_\tau(\lambda v^\tau. P))$, etc.

⁹ See also Rayo and Yablo 2001. For a competing treatment that assimilates the quantifiers in such sentences to first-order quantifiers, see Moltmann 2013: ch. 3.

But following Williamson (2003), many recent authors reject the idea that the intelligibility of the formal language depends on our ability to translate each of its sentences into a natural language. There are other routes to understanding a new constant, such as grasping its *inferential role* in the linguistic practice of those who use it: what they are happy to infer from a sentence involving the constant, and what they are happy to infer it from. In the case of the higher-order quantifiers and identity, these inferential roles are easy to master, since they are exactly the same as those of the first-order quantifiers and first-order identity, modulo the change in type. These roles are very rich. As we discuss, there is a sense in which they *completely pin down* the interpretation of the symbols that they characterize.

References

- Curry, Haskell Brooks and Feys, Robert (1958), *Combinatory Logic* (Amsterdam: North- Holland).
- Dorr, Cian (2016), "To Be F Is To Be G", in John Hawthorne and Jason Turner (eds.), *Philosophical Perspectives 30: Metaphysics* (Oxford: Blackwell), 1–97.
- Frege, Gottlob (1893), *Grundgesetze der Arithmetik: begriffsschriftlich abgeleitet* (H. Pohle).
- Moltmann, Friederike (2013), *Abstract Objects and the Semantics of Natural Language* (Oxford: Oxford University Press).
- Prior, A. N. (1971), *Objects of Thought*, ed. P. T. Geach and A. J. P. Kenny (Oxford: Oxford University Press).
- Rayo, Augustin and Yablo, Stephen (2001), "Nominalism through De-Nominalisation", *Noûs*, 35: 74–92.
- Russell, Bertrand (1908), "Mathematical Logic as Based on the Theory of Types", *American Journal of Mathematics*, 30: 222–62.
- Stalnaker, Robert C. (1977), "Complex Predicates", *The Monist*, 60: 327–39.
- Williamson, Timothy (2003), "Everything", in John Hawthorne and Dean Zimmerman (eds.), *Philosophical Perspectives 17: Language and Philosophical Linguistics* (Oxford: Blackwell), 415–65.